Day 31: RandomizedSearchCV — Smarter Hyperparameter Search

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 5, Day 4

Every grid used since Day 17 stayed deliberately small — 18 to 27 combinations — to keep GridSearchCV's exhaustive search fast. A genuinely thorough grid doesn't stay small. Today covers RandomizedSearchCV: sampling a fixed number of combinations instead of enumerating every one, verified with a same-budget, head-to-head comparison against GridSearchCV. Every topic includes the actual numbers and chart this lesson's search produced.

1. Why GridSearchCV Stops Scaling

> **Topic specification:** GridSearchCV trains and cross-validates every single combination in a parameter grid. The total number of combinations is the product of how many values each hyperparameter has — growth that compounds fast as more hyperparameters or more values per hyperparameter are added.

Explanation: A grid across 7 hyperparameters with only 3-4 values each is not an unusually large search — it's a realistic one for tuning XGBoost properly. Multiplying those value counts together produces thousands of combinations before cross-validation is even factored in.

Real-world scenario: A team tuning a production fraud model realizes their 'quick' grid search — 3 values each for 6 hyperparameters — is actually 729 combinations, and at 5-fold cross-validation with each fit taking 10 seconds, that's over 10 hours of unattended compute before results come back, discovered only after the job had already been running for an hour.

Implementation:

```
full_grid = {
    "n_estimators": [100, 200, 300],
    "learning_rate": [0.01, 0.05, 0.1, 0.2],
    "max_depth": [2, 3, 4, 5],
    "subsample": [0.6, 0.8, 1.0],
    "colsample_bytree": [0.6, 0.8, 1.0],
    "reg_alpha": [0, 0.1, 1],
    "reg_lambda": [0.5, 1, 2],
}
n_combinations = 1
for values in full_grid.values():
    n_combinations *= len(values)
total_fits = n_combinations * 5  # cv=5
```

Actual output:

Metric	Value
Hyperparameters in the grid	7

Total combinations	3,888
Cross-validation folds	5
Total model fits required	19,440

Line-by-line explanation

• `n_combinations *= len(values)` — multiplies the number of values for every hyperparameter together; this is what makes the total grow multiplicatively, not additively, as hyperparameters are added.

• `total_fits = n_combinations * 5` — each combination gets trained and validated once per cross-validation fold, so the true compute cost is the combination count times the fold count.

• Verified in this lesson: this exact 7-parameter grid comes out to 3,888 combinations and 19,440 total model fits — the motivating number for why RandomizedSearchCV exists.

Why choose this? GridSearchCV guarantees the single best combination within the grid will be found, since every combination is tried — valuable when the grid is genuinely small and compute is not a constraint.

Why NOT this (tradeoffs)? The guarantee only holds if the grid is small enough to finish in reasonable time. Once a grid grows past a handful of hyperparameters, exhaustive search becomes impractical long before it becomes impossible.

2. RandomizedSearchCV

Topic specification: `RandomizedSearchCV(pipeline, param_distributions, n_iter=N, cv=5)` samples exactly N combinations at random from the given parameter space and only trains those, instead of enumerating every possible combination.

Explanation: Because the number of combinations tried is controlled directly by n_iter, RandomizedSearchCV's compute cost is completely decoupled from how many hyperparameters or how many possible values each one has — a search across 7 continuous-valued hyperparameters costs exactly the same as a search across 3, for the same n_iter.

Real-world scenario: The same fraud-model team switches their 729-combination grid to RandomizedSearchCV with n_iter=50, getting a full search finished in under an hour instead of 10, at the cost of not being mathematically guaranteed to find the single best combination in that original 729-point grid.

Implementation:

```
search = RandomizedSearchCV(
    pipeline, param_distributions, n_iter=18, cv=5,
    n_jobs=-1, random_state=42
)
search.fit(X_train, y_train)
print(search.best_params_, search.best_score_)
```

Line-by-line explanation

• `n_iter=18` — the exact number of combinations that will be sampled and tried, regardless of how large param_distributions' underlying space actually is.

• `random_state=42` — makes which specific combinations get sampled reproducible across runs.

• `n_jobs=-1` — runs the sampled combinations' cross-validation folds in parallel across every available CPU core, same as GridSearchCV.

• `search.best_params_` / `search.best_score_` — the best of the N sampled combinations and its mean cross-validated score, read identically to GridSearchCV's output.

Why choose this? Compute cost stays fixed and predictable (n_iter combinations, no more) no matter how large or how many hyperparameters the search space has — a direct, controllable dial instead of an explosive multiplication.

Why NOT this (tradeoffs)? There's no guarantee the single best combination in the space gets sampled — a genuinely narrow optimum that a fine grid might land on exactly could be missed entirely by chance.

3. Continuous Parameter Distributions

Topic specification: `scipy.stats` distributions — `randint(low, high)`, `uniform(loc, scale)`, `loguniform(low, high)` — replace fixed lists of values, letting RandomizedSearchCV sample any value in a continuous range, not just a small hand-picked set.

Explanation: `randint(2, 8)` samples integers from 2 up to (not including) 8. `uniform(0.5, 0.5)` samples real numbers evenly across [0.5, 1.0) — the second argument is a width, not an upper bound. `loguniform(0.01, 0.3)` samples on a logarithmic scale, appropriate for learning_rate since its effect is closer to linear in log-space than in linear space.

Real-world scenario: A hand-picked learning_rate grid of [0.01, 0.05, 0.1, 0.2] might skip directly over 0.03, the value that actually performs best for a given dataset. A continuous loguniform distribution can sample 0.03, or 0.087, or any other value in between — values a fixed list could never have included by construction.

Implementation:

```
from scipy.stats import randint, uniform, loguniform

param_distributions = {
    "classifier__n_estimators": randint(100, 400),
    "classifier__max_depth": randint(2, 8),
    "classifier__learning_rate": loguniform(0.01, 0.3),
    "classifier__subsample": uniform(0.5, 0.5),
    "classifier__colsample_bytree": uniform(0.5, 0.5),
    "classifier__reg_alpha": uniform(0.0, 2.0),
    "classifier__reg_lambda": uniform(0.5, 2.0),
}
```

Line-by-line explanation

• `randint(100, 400)` — n_estimators sampled as a whole number between 100 and 399 inclusive.

• `loguniform(0.01, 0.3)` — learning_rate sampled log-uniformly, so 0.01-0.03 is as likely to be sampled from as 0.1-0.3, matching how learning rate's effect typically scales.

• `uniform(0.5, 0.5)` — subsample/colsample_bytree sampled evenly across [0.5, 1.0); note the second argument is a range WIDTH added to the first, not the upper bound itself.

• Verified in this lesson: the winning RandomizedSearchCV combination used max_depth=7 and learning_rate≈0.029 — values Topic 4's fixed grid (max_depth capped at 4, learning_rate starting at 0.05) could never have reached at all.

Why choose this? Continuous distributions can discover a genuinely better value that happens to fall between a hand-picked grid's fixed points, and they scale to covering wide, realistic ranges without needing to enumerate every value in that range.

Why NOT this (tradeoffs)? Choosing an appropriate distribution shape (uniform vs loguniform, and reasonable bounds) requires some domain knowledge of how each hyperparameter behaves — a fixed list, while coarser, is more transparent and requires no distributional assumptions.

4. A Fair, Same-Budget Comparison

Topic specification: Configuring GridSearchCV's small grid (3 hyperparameters, 18 fixed combinations) and RandomizedSearchCV's n_iter (18 sampled combinations from a 7-hyperparameter continuous space) to require the exact same number of model fits — 90 each, at cv=5 — isolates the comparison to search strategy alone.

Explanation: With identical compute budgets, any difference in the best score found or the wall-clock time taken reflects the search strategy and space, not one method simply being given more resources than the other — a controlled, apples-to-apples test.

Real-world scenario: A model-selection review comparing tuning approaches for a production pipeline runs both searches under an identical time budget before deciding which strategy to standardize on for future tuning jobs, rather than trusting either method's reputation.

Implementation:

```
# Both searches run under the same 90-fit budget:
grid_search = GridSearchCV(pipeline, small_grid, cv=5)     # 18 combos x 5
random_search = RandomizedSearchCV(
    pipeline, param_distributions, n_iter=18, cv=5)          # 18 samples x 5
```

Actual output:

search	space	fits	time_s	best_cv_score
GridSearchCV	3 params, 18 fixed combos	90	2.7	0.742
RandomizedSearchCV	7 params, continuous	90	1.8	0.737

Line-by-line explanation

• Both rows represent the identical fit budget (90 model fits) — the only variables that differ are which hyperparameters were searched and how (fixed grid vs continuous sampling).

• Verified in this lesson: GridSearchCV's narrower 3-parameter grid found a marginally better score (0.742 vs 0.737) — but RandomizedSearchCV searched more than double the hyperparameters, in less wall-clock time, for a difference of about half a percentage point.

Why choose this? For the same compute budget, RandomizedSearchCV explores a dramatically larger space — worthwhile whenever more hyperparameters genuinely matter, which is the common case for gradient boosting models.

Why NOT this (tradeoffs)? When only a handful of hyperparameters matter and their good ranges are already roughly known, a small, well-chosen grid can match or beat random sampling on the same budget, exactly as it did on best_cv_score in this lesson's comparison.

5. n_iter — the Tradeoff Dial

Topic specification: `n_iter` is the single parameter controlling how many combinations RandomizedSearchCV samples and trains — turning it up increases search thoroughness and compute cost together; turning it down decreases both, independent of how large or how many hyperparameters the space actually contains.

Explanation: Unlike a grid, where adding a hyperparameter or a value multiplies the total cost, n_iter is set directly and stays constant regardless of the space's dimensionality — the search space can be made arbitrarily large without changing the compute budget at all.

Real-world scenario: A team with a strict overnight compute window sets n_iter to whatever value fits that window exactly, then widens or narrows the parameter distributions around whatever region looks promising once results come back — a dial they control directly, rather than a budget that emerges indirectly from how many values they happened to list per hyperparameter.

Implementation:

```
# Same param_distributions, different n_iter — same code, different budget:
RandomizedSearchCV(pipeline, param_distributions, n_iter=10, cv=5)   # fast, rough
RandomizedSearchCV(pipeline, param_distributions, n_iter=100, cv=5)  # slow, thorough
```

Line-by-line explanation

• Both lines use the identical param_distributions — the search space doesn't change at all between them, only how many samples are drawn from it.

• This lesson used n_iter=18, deliberately matched to the 18-combination grid from Topic 4 for a fair comparison — in practice, n_iter is usually chosen based on the actual compute time budget available.

Why choose this? n_iter is a direct, predictable lever — doubling it roughly doubles compute time, with no surprise multiplicative blowup the way adding a hyperparameter to a grid can cause.

Why NOT this (tradeoffs)? A low n_iter on a very large or high-dimensional space may sample too sparsely to reliably find a good region at all — n_iter needs to scale at least loosely with how large and how many dimensions the search space actually has.

6. Visualizing Which Sampled Region Performed Best

Topic specification: Plotting each of the `n_iter` sampled combinations by two of their hyperparameters, colored by cross-validated score, shows visually which region of the search space the random sampling happened to land its best results in.

Explanation: `search.cv_results_` contains every sampled combination's parameters and score as a dictionary, convertible directly to a DataFrame — plotting any two parameter columns against each other, colored by score, turns that table into a visual map of the search.

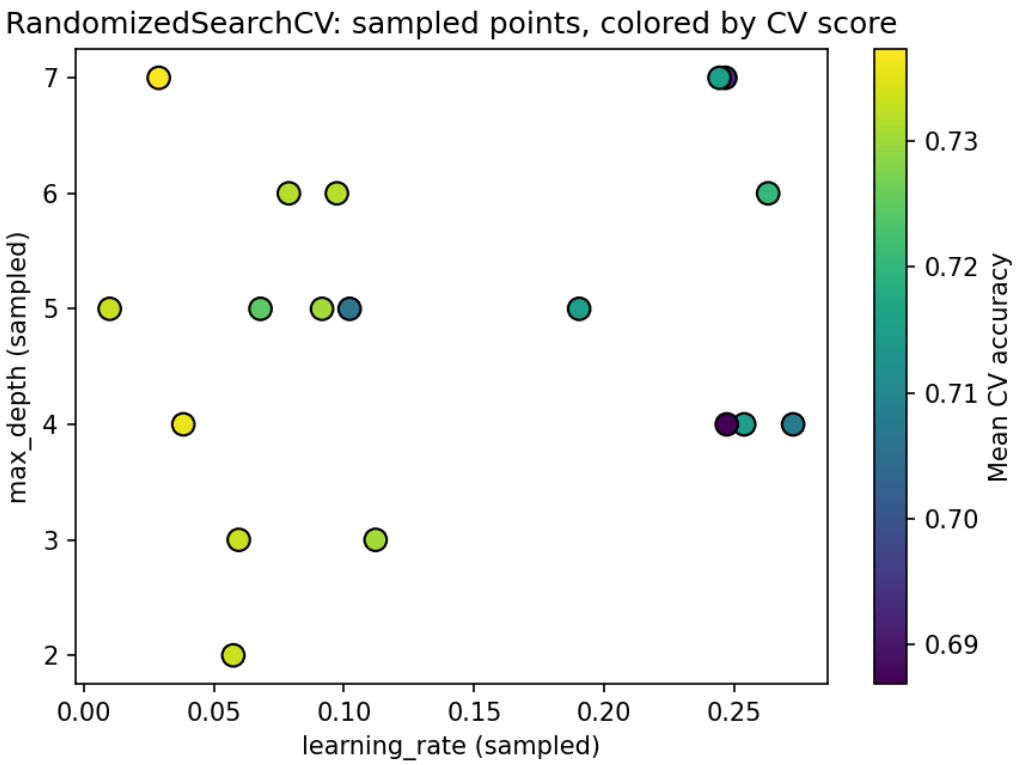
Real-world scenario: After a first RandomizedSearchCV pass, a team plots the sampled points and sees the best scores cluster around a specific learning_rate/max_depth region — they then run a focused follow-up search (a second RandomizedSearchCV or a small GridSearchCV) narrowed to just that region, refining the result without repeating the broad, expensive first pass.

Implementation:

```
results = pd.DataFrame(random_search.cv_results_)
x = results["param_classifier__learning_rate"].astype(float)
y = results["param_classifier__max_depth"].astype(float)
scores = results["mean_test_score"]

plt.scatter(x, y, c=scores, cmap="viridis", s=80, edgecolors="black")
plt.colorbar(label="Mean CV accuracy")
```

Actual output:



Actual output: randomized_search_results.png — 18 sampled points from this lesson's search.

Line-by-line explanation

• `pd.DataFrame(random_search.cv_results_)` — converts the search's raw results dictionary into a table with one row per sampled combination.

• `results["param_classifier__learning_rate"]` — cv_results_ prefixes every hyperparameter column with param_, followed by the exact pipeline step path used in param_distributions.

• `c=scores` — colors each point by its mean cross-validated score, turning the scatter plot into a visual heatmap of where the search performed best.

• Verified in this lesson: the best-scoring sampled point landed at learning_rate≈0.029, max_depth=7 — visible directly as the brightest-colored dot in the saved plot.

Why choose this? A visual map of sampled points immediately shows whether good scores cluster in one region (suggesting a focused follow-up search would help) or scatter randomly (suggesting the hyperparameters plotted don't matter much) — insight a results table alone doesn't surface as quickly.

Why NOT this (tradeoffs)? A 2D plot can only show two hyperparameters at a time — with 7 hyperparameters in this lesson's search, a single scatter plot necessarily leaves 5 dimensions unvisualized, and a good-looking region in 2D might not hold up once the other dimensions are considered.

Interview Preparation — Technical Questions

Q: Why does adding one more hyperparameter to a GridSearchCV grid affect compute cost differently than adding one more to a RandomizedSearchCV search?

A: GridSearchCV's total combinations are the product of every hyperparameter's value count, so adding one more hyperparameter multiplies the total. RandomizedSearchCV's cost is set directly by n_iter and doesn't change at all when a hyperparameter is added to param_distributions — only the space being sampled from grows, not the number of samples drawn.

Q: What does n_iter actually control in RandomizedSearchCV?

A: The exact number of hyperparameter combinations that get randomly sampled and trained, regardless of how large or how many dimensions the underlying parameter space has.

Q: Why is loguniform often used for learning_rate instead of uniform?

A: Learning rate's effect on training tends to scale multiplicatively rather than additively — the difference between 0.01 and 0.02 matters as much as the difference between 0.1 and 0.2. A log-uniform distribution samples evenly across orders of magnitude, giving small values a fair chance of being sampled instead of being crowded out by a plain uniform distribution biased toward the larger end of the range.

Q: In this lesson's same-budget comparison, why did GridSearchCV score slightly higher despite searching a much smaller space?

A: GridSearchCV concentrated its full 90-fit budget on a narrow, hand-picked 3-parameter grid already centered on reasonable values, giving it a denser search of a small promising region. RandomizedSearchCV spread the same 90 fits across a much larger 7-parameter space, sampling more thinly per dimension — a real tradeoff, not a flaw in either method.

Q: What does search.cv_results_ contain, and how is it usually inspected?

A: A dictionary with one entry per sampled or gridded combination, including its parameters (prefixed with param_) and its scores (mean_test_score, std_test_score, and per-fold scores). It's typically converted to a pandas DataFrame for filtering, sorting, or plotting.

Q: Why does uniform(0.5, 0.5) sample from [0.5, 1.0) and not from [0.5, 0.5]?

A: scipy.stats.uniform's second argument is a scale (width), added to the first argument (location), not an upper bound by itself — uniform(loc, scale) samples from [loc, loc + scale). uniform(0.5, 0.5) therefore covers [0.5, 1.0).

Q: Why is random_state set on RandomizedSearchCV specifically?

A: It controls which specific combinations get randomly sampled from the distributions, making the exact search reproducible across runs — without it, rerunning the same code could sample an entirely different set of combinations and potentially report a different best result.

Q: Why might a team run RandomizedSearchCV first and GridSearchCV second, rather than choosing only one?

A: RandomizedSearchCV can cheaply explore a wide space to identify a promising region. A focused GridSearchCV can then exhaustively search just that narrowed region at low cost, combining random search's coverage with grid search's exhaustive guarantee — but only over the smaller region the first pass identified.

Interview Preparation — Theoretical Questions

Q: Why is RandomizedSearchCV often described as making a better use of a fixed compute budget in high-dimensional search spaces, even without any guarantee of finding the true optimum?

A: In high dimensions, a grid's fixed points concentrate coverage unevenly — most of a grid's combinations end up only varying one or two hyperparameters at a time relative to their neighbors, wasting evaluations on redundant regions. Random sampling spreads evaluations more evenly across all dimensions simultaneously, which empirical studies (notably Bergstra & Bengio, 2012) showed tends to find better configurations than grid search under an equal budget, because not every hyperparameter contributes equally to performance and random sampling doesn't waste budget assuming otherwise.

Q: Why can a hyperparameter that turns out not to matter much actually make RandomizedSearchCV relatively more efficient than GridSearchCV?

A: If one hyperparameter has little effect on performance, a grid still spends a full multiplicative factor of combinations varying it for no benefit. Random sampling doesn't allocate a fixed number of trials 'wasted' on that unimportant dimension — the effective resolution on the hyperparameters that do matter ends up higher, for the same total budget, when the unimportant dimension isn't artificially inflating the combination count.

Q: Why is comparing GridSearchCV and RandomizedSearchCV under a mismatched compute budget considered a flawed benchmark?

A: If one method is allowed more model fits than the other, any advantage it shows could simply reflect having more compute, not a better search strategy. This lesson's comparison deliberately fixed both to 90 fits specifically so any difference in outcome could be attributed to the search method and space, not to an unequal budget.

Q: Why doesn't finding a slightly lower best_cv_score with RandomizedSearchCV in this lesson mean GridSearchCV is the better method in general?

A: This result reflects one specific comparison, on one dataset, with one specific choice of grid and distributions — the grid happened to be centered on values already known to work reasonably well from earlier days' tuning. On a less-understood problem, or with a genuinely wider realistic hyperparameter space, RandomizedSearchCV's broader coverage would be expected to matter more; conclusions from one comparison don't generalize automatically to every tuning problem.

Q: Why might visualizing only two of seven searched hyperparameters at a time risk a misleading conclusion about where the 'best' region of the space is?

A: A combination that looks good when only two dimensions are plotted might only perform well because of a favorable value in one of the five unplotted dimensions, not because of the two plotted

values themselves. A region that looks promising in a 2D slice could look completely different, or not exist as a coherent 'good region' at all, once every dimension is considered together.

Q: Why is the choice of distribution shape (uniform vs loguniform vs randint) itself a form of domain knowledge being encoded into the search?

A: Choosing loguniform for learning_rate encodes an assumption that its effect scales multiplicatively; choosing a narrow uniform range for subsample encodes an assumption that values outside [0.5, 1.0] aren't worth exploring. These choices shape which regions of the true space actually get sampled — a poorly chosen distribution can systematically under-sample the region where the true optimum lives, in the same way a poorly chosen grid can miss it entirely.

Q: Why does the concept of 'same-budget comparison' generalize beyond just GridSearchCV vs RandomizedSearchCV, to comparisons between any two hyperparameter tuning strategies (like Bayesian optimization)?

A: Any tuning method's apparent superiority can be an artifact of being given more compute, more iterations, or a better-informed starting point rather than a genuinely better search strategy. Controlling for compute budget — total model fits, wall-clock time, or both — is the standard way to make a fair claim that one method searches more efficiently than another, a principle that applies identically whether the second method is a grid, a random search, or a Bayesian optimizer.

Q: Why is RandomizedSearchCV considered a reasonable middle ground between GridSearchCV's exhaustiveness and Bayesian optimization's adaptivity, rather than simply an inferior shortcut?

A: GridSearchCV is exhaustive but doesn't scale; RandomizedSearchCV trades the exhaustiveness guarantee for a fixed, predictable budget across arbitrarily large spaces, while Bayesian optimization goes further by using each trial's result to intelligently choose the next point to sample, rather than sampling independently at random. RandomizedSearchCV sits between the two: no adaptivity, but no combinatorial blowup either — a legitimate, well-studied tradeoff point rather than a mere approximation of a 'real' method.

Key Takeaway and What's Next

Key takeaway: GridSearchCV and RandomizedSearchCV aren't competing answers to 'which is better' — they're different tools for different-sized search problems. A small, well-understood grid can win on a fixed budget, as it did today; a large or poorly-understood space needs RandomizedSearchCV's budget-independent scaling to be searched at all within reasonable time.

What typically follows hyperparameter search

• `Bayesian optimization (Optuna)` — instead of sampling independently at random, uses each trial's result to intelligently choose the next combination to try, converging on good regions faster than either grid or random search for the same budget.

• `Model persistence and serving` — saving the tuned pipeline with joblib and serving it behind an API, turning a tuned model into something a portfolio project can demonstrate end to end.

• `Deep learning foundations` — the same tuning principles (budget-aware search over a hyperparameter space) reappear directly when tuning a neural network's learning rate, layer sizes, and regularization.